TrafiK – Implementation scope for the Non-relational DBMS specification

Team: TrafiK (Augustinas Pletkus, Edvinas Ryliškis, Audrius Duoblys)
Language: Go
Capacity: about 20 hours per week for the whole team (about 80 hours in total)
AI policy: all code is written by team members. AI is used only to explain concepts and documentation, not to generate code.

Summary

We commit to delivering a working, crash-safe, single-file document store reachable over HTTP. This covers 9 of the 10 Must-have requirements, five of them in a simplified form described below. Secondary indexes (M5) and the Should-have and Could-have requirements are out of scope for this release.

Technology stack

Area	Choice
Language	Go, one binary for Linux, Windows and macOS
HTTP and JSON	net/http, encoding/json
On-disk format	encoding/binary (little-endian), hash/crc32, hash/fnv
File locking	gofrs/flock
Testing and CI	go test, GitHub Actions

The server is started from the command line with the database file path. It listens on 127.0.0.1:7070 by default and shuts down cleanly on Ctrl+C. The code has one Go package per package of §6.1, and dependencies point downwards only.

Functional requirements

ID	Requirement	Decision	Scope and notes
M1	Single-file paged storage	Will do	As specified: header on page 0, page size set at creation (512 B – 64 KiB, default 4 KiB), CRC-32 on every page.
M2	Flexible record layout within a page	Will do	Slot directory as in §6.7: records packed from the end of the page without padding, O(1) lookup by slot.
M3	Tombstone	Will do	Delete sets the slot flag and the tombstone counter. Page is

	deletion with page compaction		compacted when tombstones reach 70% of its slots. Overflow pages left empty are unlinked.
M4	Hash-based primary index	Simplified	As specified (FNV-1a, overflow chains, key checks), but the bucket count is fixed at creation and there is no automatic rebuild at fill ratio 0.8.
M5	Hash-based secondary indexes	Won't do	Out of scope at this capacity. All file access goes through the pager and the catalog, so secondary indexes can be added later without changing the file format.
M6	HTTP CRUD API with a query language	Simplified	All record endpoints, paging, the plan field and the error codes of §7.4. Queries support =, &, | and parentheses, but not backslash escapes or dotted nested fields. The primary index is used when the query names the primary key; otherwise it's a full scan.
M7	Multiple collections	Simplified	Collections can be created, listed and dropped at runtime and do not interfere with each other. Pages of a dropped collection are not reused (no free list), so the file does not shrink after a drop.
M8	Durable persistent storage	Will do	Provided through M9: a write is acknowledged only after its log entries are flushed to the device.
M9	Crash recovery with a write-ahead log	Simplified	Write-ahead log with commit groups, replay after an unclean stop, and a checkpoint on shutdown and at a set log size. Simplified: modified pages stay in memory until the checkpoint, and crash tests cover a kill before and after the commit only.
M10	Cross-platform support	Simplified	Fixed-width little-endian integers and UTF-8 text. Tests run in CI on Linux; Windows and macOS builds are cross-compiled in CI but not tested there.
S1	Multiple concurrent readers	Won't do	All operations are serialised by one lock, as NFR 5.3 sets for the first release. All file access goes through the pager, so readers can be added later.
S2	Result caching	Won't do	Out of scope for this release.
C1	Data compression	Won't do	Out of scope for this release.
C2	Slotted array optimisation	Won't do	The M2 slot directory already gives O(1) lookup. The requirement does not state a measurable target.

Operational endpoints: GET /health as specified. GET /stats in reduced form, without cache hit rate or load factors.

Non-functional requirements

- **Performance (5.1):** measured and reported, not guaranteed. We will run a short benchmark with

100,000 records and report the results. With all writes serialised and flushed to the device, the 1,000 durable writes per second target depends on the storage device and OS.

- **Durability and consistency (5.2):** met. Each request is atomic, including a DELETE that matches many records.
- **Concurrency (5.3):** met. One operation at a time, concurrent connections are queued, and a second process is refused by the file lock.
- **Capacity and limits (5.4):** the limits in the table are enforced with their status codes. Reads use the OS file cache instead of an LRU page cache, so the 128 MiB bound is not guaranteed; memory grows only up to the log-size threshold that triggers a checkpoint.
- **Reliability, observability and security (5.5):** met. Invalid input returns an error instead of crashing the server, the file and log are checked by magic value, version and checksum, every request is logged with its plan, recovery is logged, and the server binds to loopback.

Plan

- **Week 1 (16–22 Nov):** agree on-disk formats and package interfaces. Codec (little-endian encoding, CRC-32, FNV-1a, canonical text), pager with the file header and file lock, slotted pages with tombstones. CI with Linux tests and Windows and macOS builds. About 19 hours.
- **Week 2 (23–29 Nov):** catalog and collections, primary index, compaction, query parser, HTTP skeleton. Goal: insert, read, update and delete work end to end, still without the log. About 20 hours.
- **Week 3 (30 Nov – 6 Dec):** write-ahead log with commit groups and replay. Query planner (primary key or full scan), error codes and limits. About 20 hours.
- **Week 4 (7–14 Dec):** checkpoints, crash tests, integration tests and a short benchmark. About 9 hours, with about 11 hours kept as buffer.

Work is split into three areas, each with one owner: storage and durability (pager, slotted pages, write-ahead log), data and indexing (codec, catalog, primary index), and interface and quality (query language, HTTP API, CI, tests, benchmarks). Every change is reviewed by another team member before merging.